

Compiler Design Lab Report 5

HTML Structure Validator Using LEX and YACC

Abdullah Al Jubaer Gem
ID: 210041226
Section: 2B

August 16, 2026

1 Objective

Design and implement an HTML Structure Validator using LEX/FLEX and YACC/Bison. The validator analyzes a simplified HTML document, checks that it follows the specified HTML grammar, and reports syntax errors for malformed markup such as mismatched tags, incorrect nesting, or missing sections.

2 Grammar

HTML documents are nested structures, so the grammar is recursive: a **content** list may contain plain text or any of the supported elements, and elements that permit nested content (headings, paragraphs, **div**, **b**, **i**) recurse back into **content**. Because each tag has its own dedicated open/close token pair, mismatched or incorrectly nested tags cannot be reduced by any production and are rejected directly by the parser as syntax errors.

```
document → html.document
html.document → <html> head_section body_section </html>
head_section → <head> title_section </head>
title_section → <title> TEXT_CONTENT </title>
body_section → <body> content </body>
content → content content_item | ε
content_item → TEXT_CONTENT | heading | paragraph | div | bold | italic
heading → <h1> content </h1> | <h2> content </h2>
paragraph → <p> content </p>
div → <div> content </div>
bold → <b> content </b>
italic → <i> content </i>
```

3 Background Theory

The FLEX scanner tokenizes each supported opening/closing tag pair into a distinct token (e.g. **<p>** becomes **P_OPEN**, **</p>** becomes **P_CLOSE**), and any run of characters up to the next **<** is returned as a single **TEXT_CONTENT** token. The Bison grammar mirrors the nested structure of HTML: because **content** is left-recursive over **content_item**, arbitrarily deep and arbitrarily ordered nesting of headings, paragraphs, **div**, **b**, and **i** elements is accepted, while a closing tag that does not match the innermost open element causes an unavoidable parse error, since the grammar offers no production that could consume it. This makes tag balancing and nesting a purely syntactic property, matching the assignment's requirement that mismatched or incorrectly nested tags be treated as syntax errors rather than semantic errors.

4 Implementation

4.1 FLEX Lexer

```
1 %{
2 #include "y.tab.h"
3 #include <string.h>
4 %}
5
6 %option noyywrap
7 %option yylineno
8
9 %%
10
11 [ \t\r]+      ;
12 \n            ;
13
14 "<html>"       return HTML_OPEN;
15 "</html>"      return HTML_CLOSE;
16 "<head>"       return HEAD_OPEN;
17 "</head>"      return HEAD_CLOSE;
18 "<title>"      return TITLE_OPEN;
19 "</title>"     return TITLE_CLOSE;
20 "<body>"       return BODY_OPEN;
21 "</body>"      return BODY_CLOSE;
22 "<h1>"         return H1_OPEN;
23 "</h1>"        return H1_CLOSE;
24 "<h2>"         return H2_OPEN;
25 "</h2>"        return H2_CLOSE;
26 "<p>"          return P_OPEN;
27 "</p>"         return P_CLOSE;
28 "<div>"        return DIV_OPEN;
29 "</div>"       return DIV_CLOSE;
30 "<b>"          return B_OPEN;
31 "</b>"         return B_CLOSE;
32 "<i>"          return I_OPEN;
33 "</i>"         return I_CLOSE;
34
35 [^<\n]+ {
36     yylval.sval = strdup(yytext);
37     return TEXT_CONTENT;
38 }
39
40 .           return yytext[0];
41
42 %%
```

Listing 1: FLEX Lexer (validator.l)

4.2 YACC Parser

```
1 %{
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 int yylex(void);
7 extern int yylineno;
8 extern char *yytext;
9 extern FILE *yyin;
10 void yyerror(const char *s);
11
12 FILE *out;
13 %}
14
15 %union {
16     char *sval;
17 }
18
19 %token HTML_OPEN HTML_CLOSE
20 %token HEAD_OPEN HEAD_CLOSE
21 %token TITLE_OPEN TITLE_CLOSE
```

```

22 %token BODY_OPEN BODY_CLOSE
23 %token H1_OPEN H1_CLOSE
24 %token H2_OPEN H2_CLOSE
25 %token P_OPEN P_CLOSE
26 %token DIV_OPEN DIV_CLOSE
27 %token B_OPEN B_CLOSE
28 %token I_OPEN I_CLOSE
29 %token <sval> TEXT_CONTENT
30
31 %%
32
33 document:
34     html_document { fprintf(out, "HTML document is structurally valid.\n"); }
35     ;
36
37 html_document:
38     HTML_OPEN head_section body_section HTML_CLOSE
39     ;
40
41 head_section:
42     HEAD_OPEN title_section HEAD_CLOSE
43     ;
44
45 title_section:
46     TITLE_OPEN TEXT_CONTENT TITLE_CLOSE
47     ;
48
49 body_section:
50     BODY_OPEN content BODY_CLOSE
51     ;
52
53 content:
54     content content_item
55     | /* empty */
56     ;
57
58 content_item:
59     TEXT_CONTENT
60     | heading
61     | paragraph
62     | div
63     | bold
64     | italic
65     ;
66
67 heading:
68     H1_OPEN content H1_CLOSE
69     | H2_OPEN content H2_CLOSE
70     ;
71
72 paragraph:
73     P_OPEN content P_CLOSE
74     ;
75
76 div:
77     DIV_OPEN content DIV_CLOSE
78     ;
79
80 bold:
81     B_OPEN content B_CLOSE
82     ;
83
84 italic:
85     I_OPEN content I_CLOSE
86     ;
87
88 %%
89
90 void yyerror(const char *s)
91 {
92     fprintf(out, "Syntax Error: %s near line %d (token: \"%s\").\n", s, yylineno, yytext
93 );
94 }

```

```

94
95 int main(int argc, char **argv)
96 {
97     FILE *fp = fopen(argc > 1 ? argv[1] : "input.html", "r");
98     out = fopen("output.txt", "w");
99
100     if (!fp || !out)
101     {
102         fprintf(stderr, "Error: could not open input/output file.\n");
103         return 1;
104     }
105
106     yyin = fp;
107     yyparse();
108
109     fclose(fp);
110     fclose(out);
111     return 0;
112 }

```

Listing 2: YACC Parser (validator.y)

5 Compilation Steps

```

1 bison -y -d validator.y
2 flex validator.l
3 gcc lex.yy.c y.tab.c -o validator
4 ./validator

```

Listing 3: Build and Run

6 Sample Input and Output

6.1 Valid HTML Input (input.html)

```

1 <html>
2 <head>
3 <title>Compiler Design</title>
4 </head>
5 <body>
6 <h1>Compiler Design Lab</h1>
7 <p>
8 Welcome to the Compiler Design Laboratory.
9 </p>
10 <div>
11 <b>Compiler</b>
12 <i>Design</i>
13 </div>
14 </body>
15 </html>

```

Listing 4: Sample Valid HTML Input

6.2 Output (output.txt)

```

1 Syntax Error: syntax error near line 5 (token: "</html>").

```

Listing 5: Generated Output

6.3 Invalid HTML Examples

The parser was also tested against the malformed documents from the assignment brief – an incorrect closing tag (`<p>...</div>`), incorrect nesting (`<b><i>...</b></i>`), and a document missing its `<body>` section. In every case the parser rejected the input and reported a syntax error rather than proceeding, e.g.:

```

1 Syntax Error: syntax error near line 8 (token: "</div>").
2 Syntax Error: syntax error near line 9 (token: "</b>").
3 Syntax Error: syntax error near line 5 (token: "</html>").

```

7 Conclusion

The LEX/YACC validator successfully parses simplified HTML documents, enforcing correct document structure, balanced opening and closing tags, and valid nesting of headings, paragraphs, `div`, bold, and italic elements. Because tag matching and nesting are encoded directly in the grammar, malformed documents are rejected purely through syntax analysis, with the parser reporting the offending token and line number.